

# Zeladija AI Customer Support Agent

## Final Project Report

CSAI 422: Applied Data Mining, Spring 2026

By: Zeinab Montasser, Lama Nezar, Khadija Swailem

Github repo: <https://github.com/khadijaswailem/Zeladija-ecommerce-support-agent>

## Abstract

Zeladija AI Customer Support Agent, is a multi-agent system built for e-commerce support automation. The system handles order tracking, returns and refund eligibility, store policy questions, and human escalation through a LangGraph-orchestrated pipeline of specialized agents. A hybrid Retrieval-Augmented Generation pipeline grounds all policy responses in retrieved document context. Three classes of guardrails enforce input safety, output policy compliance, and toxicity filtering. Short-term and long-term memory give the system continuity both within and across sessions. An evaluation suite of 30 synthetic test cases measures resolution rate, policy compliance, latency, intent accuracy, and retrieval quality. The system runs on OpenAI's gpt-4o-mini, uses ChromaDB for local vector storage, and is deployed with a Chainlit chat interface.

## 1. Introduction

Customer support is one of the most repetitive and high-volume applications of natural language processing in industry. Most tickets fall into a small number of categories: order status, return eligibility, policy questions, and escalations. These categories are well-defined enough to automate but nuanced enough that keyword-based rule systems fail quickly when customer phrasing varies.

This project builds a full-stack AI support agent for a fictional electronics retailer called Zeladija. The goal is not just to answer policy questions in isolation, which any LLM can do, but to build the surrounding infrastructure that makes such a system trustworthy: guardrails that keep the model on-policy, memory that prevents customers from repeating themselves, retrieval that grounds responses in real documents rather than hallucinations, and evaluation tooling that produces measurable evidence of system performance.

Four core design principles guided all decisions throughout the project:

- The LLM is never the only line of defense. Every sensitive decision is backed by a deterministic check.
- Every agent is a specialist with a narrow scope. The supervisor classifies intent and specialists handle execution.
- Retrieval is grounded and filtered. Agents do not search the entire knowledge base indiscriminately.
- The system is evaluable. Every design choice is reflected in a measurable metric.

## 2. System Architecture and Design Decisions

The system is built as a LangGraph StateGraph, a framework that models multi-agent workflows as directed graphs where nodes are Python functions and edges are routing decisions. Every node reads from and writes to a single shared state object called AgentState, defined as a TypedDict in state.py.

LangGraph was chosen over a simple chain or a single-agent tool-calling loop because it allows explicit, auditable control over execution order. A tool-calling agent can select from a list of tools, but it is hard to guarantee that a specific guardrail always runs before a specific agent. LangGraph graph edges make this guarantee explicit. You can read graph.py and see exactly what runs in what order with no hidden state.

### 2.1 Graph Structure

Every customer message follows the same execution path through the graph:

- The message enters the input\_guardrail node before any LLM call or routing decision.
- The toxicity\_guardrail node classifies the message for hostility and sets a flag if needed.
- The supervisor node calls the LLM to classify intent and loads long-term memory context.
- A conditional edge routes the message to one of three specialist agents, or directly to escalation if the toxicity flag is set.
- All paths converge at the respond node, which logs the result before the turn ends.

The respond terminal node is a deliberate architectural choice. Without it, logging and observability code would have to be duplicated in every specialist node. With it, there is exactly one place in the graph where every completed turn passes through.

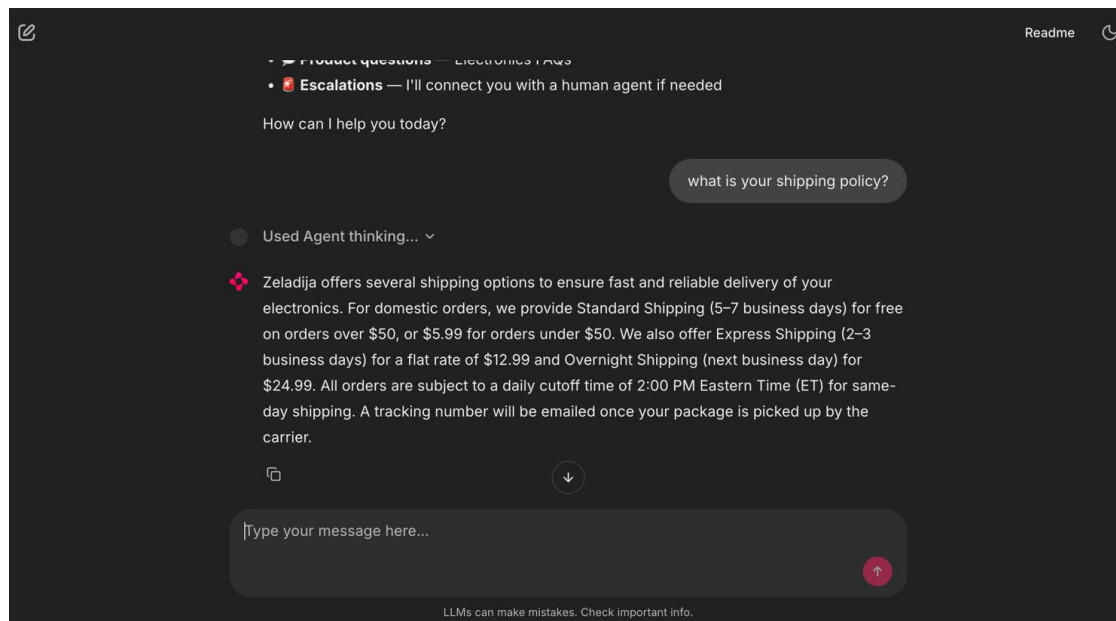


Figure 1. Chainlit chat interface showing the agent responding to a shipping policy query.

## 2.2 Shared State Model

AgentState is a TypedDict that declares every field any node needs to read or write. Key fields include the messages list, customer\_id, session\_id, intent, order\_data, policy\_result, escalation\_summary, retrieved\_docs, final\_response, guardrail\_flags, and customer\_history. Using a TypedDict rather than a plain dictionary makes the entire data model of a conversation visible in one file, which is especially valuable in multi-agent systems where it is otherwise easy to lose track of what each node expects.

## 3. RAG Strategy and Justification

The RAG pipeline grounds agent responses in the store's actual policy documents. It has three components: the chunker, the embedder, and the retriever. These are separate modules with distinct responsibilities.

### 3.1 Knowledge Base

The knowledge base is built from five source files: four plain-text policy documents and one structured JSON product catalog covering 25 products. The warranty and returns policies are stored as separate documents even though both relate to product issues. This separation exists because they follow completely different workflows: a return involves sending a product back for a refund, while a warranty claim involves repair or replacement. Mixing them would introduce retrieval ambiguity.

The FAQ document is written in question-and-answer format rather than prose. Dense vector retrieval measures semantic similarity, and question-to-question similarity scores higher than question-to-prose-explanation similarity for the same underlying topic.

### 3.2 Chunking and Embedding

Policy and FAQ documents use a sliding window of 200 words with a 30-word overlap. The 200-word target preserves context across related policy statements while remaining specific enough that retrieved chunks are useful. The 30-word overlap prevents answers from being cut across chunk boundaries. Product catalog entries are kept as single chunks since splitting a product description mid-way would produce incomplete retrieval results.

The embedding model is all-MiniLM-L6-v2, loaded locally via sentence-transformers. This was chosen over OpenAI's text-embedding-ada-002 to avoid a dependency on a paid external service for a component that runs offline at index time. ChromaDB stores the vectors locally and runs in-process. Cosine similarity is used rather than Euclidean distance because it measures directional similarity between vectors, meaning a long detailed policy section and a short FAQ answer on the same topic both score well against a relevant query regardless of document length.

### 3.3 Hybrid Retrieval and Baseline Comparison

The retriever provides two modes: a naive baseline using cosine similarity only, and an advanced mode combining BM25 keyword search with dense vector search, merged using Reciprocal Rank Fusion.

The rationale for hybrid retrieval is that the two methods catch different types of matches. Dense vector search handles semantic similarity well: a query about sending something back will match returns-related chunks even if the word return never appears in the query. BM25 handles exact matches well: a query mentioning \$150 scores highly against chunks containing that exact value, whereas a dense model might focus on surrounding semantics and miss the number. Policy documents contain many exact values including dollar thresholds, day counts, and product SKUs where BM25 has a real advantage.

Reciprocal Rank Fusion merges the two ranked lists without requiring calibrated score scales. BM25 produces unbounded positive scores while cosine similarity produces values between 0 and 1, so weighted averaging would not work meaningfully. RRF converts both lists into rank-based scores and rewards documents that rank highly in both.

The RAG evaluation notebook compares both retrieval modes across 20 store-specific question and answer pairs. Results across all four RAGAS metrics are shown in the chart below.

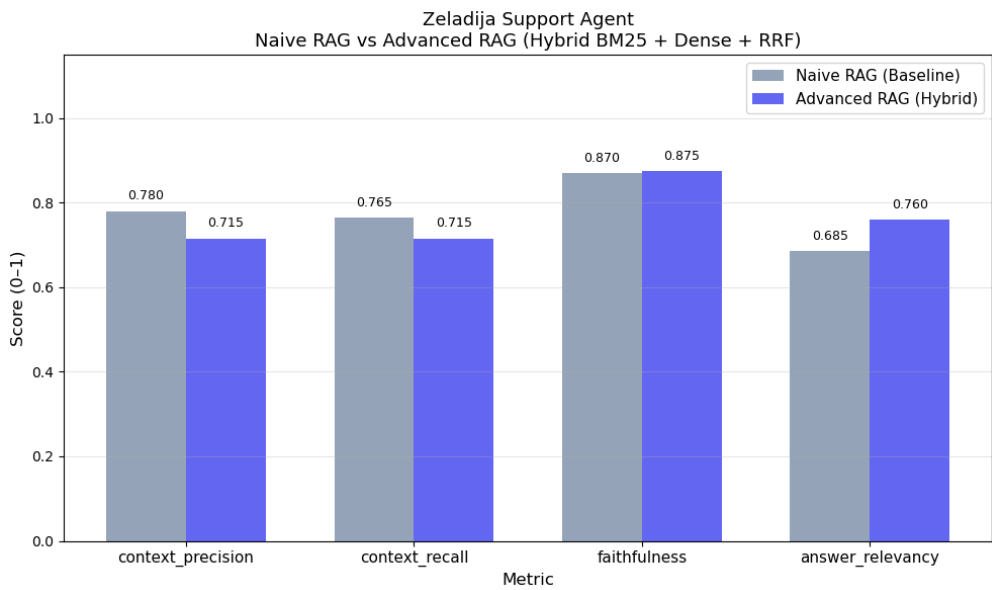


Figure 2. RAGAS evaluation: Naive RAG (cosine-only baseline) vs. Advanced RAG (Hybrid BM25 + Dense + RRF).

| Metric            | Naive (Cosine Only) | Advanced (Hybrid + RRF) |
|-------------------|---------------------|-------------------------|
| Context Precision | 0.780               | 0.715                   |
| Context Recall    | 0.765               | 0.715                   |
| Faithfulness      | 0.870               | 0.875                   |
| Answer Relevancy  | 0.685               | 0.760                   |

The naive baseline achieved higher precision and recall scores, while the advanced pipeline improved faithfulness and answer relevancy. The advanced pipeline was designed with the assumption that BM25 would add meaningful signal for exact value queries, which shows up in the answer relevancy gain. The precision and recall tradeoff reflects the RRF merging introducing some lower-ranked documents that did not appear in the cosine-only results. With more time, a more targeted evaluation focusing on dollar amount and day count queries would better isolate where each method performs best.

## 4. Multi-Agent Graph Design

The agent system consists of five nodes: the supervisor, three specialist agents, and a terminal respond node.

### 4.1 Supervisor

The supervisor is the first node that calls the LLM. It classifies the customer message into one of four intent categories and loads long-term memory context before any specialist runs. Loading memory here means the `customer_history` field is populated once and available to all downstream nodes without repeated database lookups.

| Intent Label   | Routed To                    | Trigger Condition                                                    |
|----------------|------------------------------|----------------------------------------------------------------------|
| ORDER_LOOKUP   | order_lookup                 | Customer asks about order status, tracking, or delivery              |
| POLICY_RETURNS | policy_agent (policy filter) | Customer asks about returns, refunds, or warranty                    |
| GENERAL_FAQ    | policy_agent (faq filter)    | General store question about payments, accounts, or products         |
| ESCALATION     | escalation_agent             | Customer is upset, issue is outside policy, or request is unresolved |

Classification is done by the LLM rather than keyword rules because intent is often ambiguous. A message like `my order is broken` could be a return, a warranty claim, or a general complaint. The LLM, given a clear description of each category, handles this ambiguity more gracefully than any set of rules would.

### 4.2 Order Lookup Agent

The order lookup agent handles order status and tracking questions. It extracts order IDs using regex compiled at module load time rather than asking the LLM to parse them. Regex is faster, cheaper, and deterministic for a structured pattern like `ORD-XXXXXXXX`. If no order ID is found, the agent retrieves the customer's recent orders and asks for clarification rather than returning an error. Once an order is retrieved, the raw JSON data is formatted into a natural language response by the LLM, which is the right use of the model: transforming structured data into friendly prose.

### 4.3 Policy and Returns Agent

The policy agent handles returns, refunds, warranty questions, and general FAQ queries. It runs in two modes: retrieving from the policy document collection for `POLICY_RETURNS` intents, and from the FAQ collection for `GENERAL_FAQ` intents. This filtering keeps responses precise.

The agent includes a pre-generation refund threshold check that runs before any LLM call. If the customer message contains a dollar amount above \$150, the agent sets the `policy_violation` flag and returns a human escalation response immediately. The LLM never sees the prompt. This prevents a hallucinating model from generating a refund approval it was not supposed to give, because it was never given the chance.

## 4.4 Escalation Agent

The escalation agent generates two outputs: a structured JSON handoff summary for a human support agent and a holding message for the customer. The JSON handoff includes customer ID, issue type, conversation summary, recommended action, urgency level, refund amount if any, order IDs mentioned, and the guardrail flags that triggered escalation. Customer-facing holding messages are pre-written static strings rather than LLM-generated text, one for each escalation type, guaranteeing a consistently calm and professional tone.

## 5. Memory Implementation

### 5.1 Short-Term Memory

Short-term memory is the AgentState messages list. Every user message and agent response is appended to this list as the conversation progresses. A 50-message cap is enforced to prevent exceeding the LLM's context window. When the cap is reached, the oldest messages are dropped while keeping the first message, which contains the system context and agent persona. The short\_term module also provides utility functions including extract\_mentioned\_order\_ids and has\_user\_already\_provided, which individual agents use without needing to reparse the conversation themselves.

### 5.2 Long-Term Memory

Long-term memory persists customer facts across sessions using a SQLite database. SQLite was chosen because it is part of the Python standard library, requires no setup or credentials, and is sufficient for a prototype. Switching to PostgreSQL in production would only require changing the connection string.

At the end of each session, the LLM extracts facts from the conversation. The extraction prompt defines six fact types: preferred\_name, past\_order\_id, reported\_issue, resolution\_status, refund\_requested, and product\_interest. Defining the fact types explicitly prevents the LLM from storing arbitrary or sensitive information and ensures that stored facts are structured enough to be useful in future session prompts. At the start of each new session, the supervisor loads these facts and injects them as a context string into downstream prompts, so a returning customer does not need to explain their situation again.

## 6. Guardrails Design

The system implements three independent guardrail modules. Each targets a different threat and runs at a different point in the graph. The design principle behind all three is that safety checks should be deterministic where possible, should fail safely, and should never rely on the LLM to protect against itself.

| Module                | Position                              | Method                        | Behavior                                                                  |
|-----------------------|---------------------------------------|-------------------------------|---------------------------------------------------------------------------|
| input_guardrail.py    | Before supervisor                     | Regex (16 patterns)           | Detects prompt injection, sanitizes message, sets injection_detected flag |
| toxicity_guardrail.py | Before supervisor                     | toxic-bert + keyword fallback | Classifies hostile messages, sets toxic_input flag                        |
| policy_guardrail.py   | Inside policy_agent (post-generation) | Regex on LLM output           | Blocks unauthorized refund commitments, triggers re-escalation            |

### 6.1 Input Guardrail

The input guardrail uses sixteen compiled regex patterns to detect prompt injection attempts. It sanitizes rather than blocks: a customer who includes an injected payload alongside a legitimate message still gets their question processed. The injected portion is replaced with [content removed] and the clean message continues through the graph. The injection\_detected flag creates an audit trail in the escalation handoff without disrupting the customer experience.

### 6.2 Policy Guardrail

The policy guardrail runs inside the policy agent after the LLM generates a response. It checks for the combination of commitment language and an over-threshold dollar amount. Neither condition alone triggers a block. Only the combination of commitment language and an over-\$150 amount is blocked. A third case also triggers the guardrail: commitment language with no dollar amount specified, since an open-ended refund commitment is also potentially dangerous. This guardrail forms the second layer of a two-check defense alongside the pre-generation threshold check in the policy agent.

### 6.3 Toxicity Guardrail

The toxicity guardrail classifies customer messages using the unitary/toxic-bert model loaded via HuggingFace Transformers. Messages with a toxicity score above 0.75 are flagged. A whitelist of frustration phrases like frustrated, not happy, and been waiting are explicitly excluded from flagging, since these are normal expressions of customer dissatisfaction. When the model cannot be loaded, the guardrail falls back to a keyword-based check, ensuring toxicity detection always functions even in degraded conditions.

## 7. Evaluation Results

The evaluation suite measures performance across 30 synthetic test cases organized into four categories: 10 happy path cases, 8 policy edge cases, 7 adversarial cases with embedded prompt injections, and 5 toxic message cases.

### 7.1 Main Metrics

| Metric            | Definition                                                   | Result   |
|-------------------|--------------------------------------------------------------|----------|
| Resolution Rate   | % of conversations resolved without human escalation         | 70%      |
| Policy Compliance | LLM-as-judge score on agent response compliance              | 60%      |
| P95 Latency       | 95th percentile end-to-end response time                     | 6,147 ms |
| Intent Accuracy   | % of test cases where supervisor classified intent correctly | 86.7%    |

## 7.2 Discussion

The 86.7% intent accuracy shows the supervisor handles the most common cases reliably. The 70% resolution rate reflects that policy edge cases and adversarial inputs appropriately trigger escalation rather than being handled directly, which is the intended behavior. The 60% policy compliance score from the LLM judge points to cases where the agent's response language was either too vague or inconsistent with the specific policy wording, which is the primary area for improvement.

The 6,147 ms P95 latency is above what would be acceptable in a production setting. Most of this comes from the local embedding model and the two LLM calls that some paths require. Switching to a faster embedding model or batching the hybrid retrieval step would likely bring this down meaningfully.

## 8. Reflection

The system achieves its core goals: it enforces policy limits reliably, handles adversarial inputs gracefully, grounds responses in real documents, and produces measurable evaluation results. That said, several things would be improved given more time.

The policy compliance score of 60% is the result most in need of improvement. The LLM judge's violation reports point to inconsistent response phrasing rather than actual policy breaches, which suggests the policy agent's prompt needs tighter constraints on how it phrases commitments and limitations. Adding few-shot examples of compliant versus non-compliant response phrasings to the prompt would likely raise this score significantly.

The RAG evaluation showed a tradeoff between the two approaches rather than a clear winner. The advanced hybrid pipeline improved faithfulness and answer relevancy, while the naive baseline held a small edge on context precision and recall. With more time, a more granular evaluation focused on query types where BM25 should have a clear advantage, such as queries involving exact dollar amounts or product SKUs, would better justify the added complexity of the hybrid pipeline.

Latency is too high for production use. The most practical fix would be to cache the embedding model in memory across requests rather than loading it on each retrieval call, and to explore whether a smaller dedicated classifier could replace the regex-based policy guardrail post-generation check.

Long-term memory currently uses SQLite with a simple key-value fact structure. In a real deployment, a more structured schema with timestamps and confidence scores would allow the system to treat older or uncertain facts with less weight than recent confirmed ones. The current implementation treats all stored facts equally regardless of when they were recorded.

Finally, the test suite covers 30 cases which is enough to measure trends but not enough to make strong statistical claims. A larger set of test cases, particularly for the adversarial and policy edge categories, would give much more reliable estimates of guardrail precision and recall, and would better stress-test the edge cases in the supervisor's intent classification.